

Reviewer Pack — Free Entropy Gap in Read-Twice Branching Programs for IP_2

Entry aad5b9e6d7b1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 15:43:19 UTC

Free Entropy Gap in Read-Twice Branching Programs for IP_2

Entry ID: aad5b9e6d7b1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-29 15:43:19 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For any read-twice BP P computing IP_2 , the free entropy $\rho(P) = \Theta(\log n)$, whereas the trivial read-twice BP for IP_2 has $\rho(P) = \Omega(n^2)$.

Rationale (proposer's reasoning):

Free entropy measures non-commutative randomness, capturing structural constraints in BPs. Read-twice BPs have limited state reuse, while IP_2 's inherent correlation requires exponential resources, creating a gap detectable via free probability invariants.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 912a025032b708bb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.85	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.85	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_read_twice_bp(n):
        bp = []
        for i in range(2**n):
            row = [random.choice(range(2)) for _ in range(n)]
            bp.append(row)
        return bp

    def transition_matrix(bp):
        n = len(bp[0])
        M = [[0] * (2**n) for _ in range(2**n)]
        for i in range(2**n):
            for j in range(n):
                if bp[i][j] == 1:
                    M[i][i ^ (1 << j)] += 1
        return M

    def r_transform(M):
        n = len(M)
        R = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i, n):
                if i == j:
                    R[i][j] = Fraction(1, 2) * (M[i][i] + M[j][j])
                else:
                    R[i][j] = Fraction(M[i][j], 2)
        return R

    def free_entropy(R):
        n = len(R)
        det_R = 1
        for i in range(n):
            det_R *= R[i][i]
        return -math.log(det_R, 2)

    n = 40
    read_twice_bp = generate_read_twice_bp(n)
    read_twice_matrix = transition_matrix(read_twice_bp)
    R = r_transform(read_twice_matrix)
```

```

rho = free_entropy(R)

trivial_rho = math.log(n)

return {
    "metric_name": "free_entropy",
    "metric_value": rho,
    "instances_tested": 1,
    "conjecture_holds": rho >= trivial_rho - 0.1 and rho <= trivial_rho + 0.1,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 31)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rho = sum(r["metric_value"] for r in results) / len(results)
    std_rho = math.sqrt(sum((r["metric_value"] - mean_rho)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rho} std={std_rho} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"free_entropy_out_of_bounds\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run with optimized parameters for larger instance sizes

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	56211
2	preregistration	ollama_remote	qwen3:8b	0	0	24013
3	novelty	ollama_remote	qwen3:8b	0	0	20606
4	novelty	ollama_remote	qwen3:8b	0	0	15187
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20366
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8270
7	judge	ollama_remote	qwen3:8b	0	0	62057

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 206711 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/aad5b9e6d7b1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/aad5b9e6d7b1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/aad5b9e6d7b1.tar.gz` (if generated)